



UAdeO
UNIVERSIDAD AUTÓNOMA DE OCCIDENTE

6to Semestre ISOF

Asignatura: Desarrollo de Proyectos de TI

Profesor:

Honorio Haro Hereida

Alumno y Matrícula:

Jaziel Aguirre (23030588), Karen Liera (23030940), Bruno Cota
(23030461), Neyra Lopez (23030589)

Trabajo:

“Script Base de Datos”

Guasave, Sinaloa a 25 de abril de 2026

Script

```
-- =====  
  
-- TABLA: usuarios  
  
-- =====  
  
CREATE TABLE usuarios (  
    id_usuario SERIAL PRIMARY KEY,  
    nombre VARCHAR(100) NOT NULL,  
    email VARCHAR(150) UNIQUE NOT NULL,  
    contrasena TEXT NOT NULL,  
    tipo_usuario VARCHAR(20) DEFAULT 'cliente',  
    estado VARCHAR(20) DEFAULT 'activo', -- Implementación de borrado  
lógico  
    fecha_registro TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);  
  
-- =====  
  
-- TABLA: tipos_clase  
  
-- =====  
  
CREATE TABLE tipos_clase (  
    id_tipo SERIAL PRIMARY KEY,  
    nombre VARCHAR(50) NOT NULL UNIQUE,  
    cupo_maximo INT NOT NULL CHECK (cupo_maximo > 0),  
    estado VARCHAR(20) DEFAULT 'activo'
```

);

-- Insertar tipos base

INSERT INTO tipos_clase (nombre, cupo_maximo) VALUES

('Yoga', 15),

('Barre', 15),

('Pilates', 5);

-- =====

-- TABLA: clases

-- =====

CREATE TABLE clases (

id_clase SERIAL PRIMARY KEY,

id_tipo INT NOT NULL,

fecha DATE NOT NULL,

hora_inicio TIME NOT NULL,

hora_fin TIME NOT NULL,

estado VARCHAR(20) DEFAULT 'activa',

CONSTRAINT fk_tipo_clase

FOREIGN KEY (id_tipo) REFERENCES tipos_clase(id_tipo)

ON DELETE RESTRICT -- Reemplazo de CASCADE por seguridad de auditoría

);

```
-- =====
```

```
-- TABLA: reservas
```

```
-- =====
```

```
CREATE TABLE reservas (
```

```
    id_reserva SERIAL PRIMARY KEY,
```

```
    id_usuario INT NOT NULL,
```

```
    id_clase INT NOT NULL,
```

```
    fecha_reserva TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
```

```
    estado VARCHAR(20) DEFAULT 'activa',
```

```
    CONSTRAINT fk_usuario
```

```
        FOREIGN KEY (id_usuario) REFERENCES usuarios(id_usuario)
```

```
        ON DELETE RESTRICT,
```

```
    CONSTRAINT fk_clase
```

```
        FOREIGN KEY (id_clase) REFERENCES clases(id_clase)
```

```
        ON DELETE RESTRICT,
```

```
    CONSTRAINT unique_reserva UNIQUE (id_usuario, id_clase)
```

```
);
```

```
-- =====
```

```
-- TABLA: paquetes
```

-- =====

```
CREATE TABLE paquetes (  
    id_paquete SERIAL PRIMARY KEY,  
    nombre VARCHAR(100) NOT NULL,  
    cantidad_clases INT NOT NULL CHECK (cantidad_clases > 0),  
    precio NUMERIC(10,2) NOT NULL,  
    vigencia_dias INT NOT NULL CHECK (vigencia_dias > 0),  
    estado VARCHAR(20) DEFAULT 'activo'  
);
```

-- =====

-- TABLA: usuario_paquetes

-- =====

```
CREATE TABLE usuario_paquetes (  
    id_usuario_paquete SERIAL PRIMARY KEY,  
    id_usuario INT NOT NULL,  
    id_paquete INT NOT NULL,  
    clases_restantes INT NOT NULL,  
    fecha_inicio DATE NOT NULL,  
    fecha_expiracion DATE NOT NULL,  
    estado VARCHAR(20) DEFAULT 'activo',  
  
    CONSTRAINT fk_usuario_paquete_usuario  
        FOREIGN KEY (id_usuario) REFERENCES usuarios(id_usuario)
```

```

        ON DELETE RESTRICT,

CONSTRAINT fk_usuario_paquete_paquete

    FOREIGN KEY (id_paquete) REFERENCES paquetes(id_paquete)

    ON DELETE RESTRICT

);

-- =====

-- TABLA: uso_paquete

-- =====

CREATE TABLE uso_paquete (

    id_uso SERIAL PRIMARY KEY,

    id_usuario_paquete INT NOT NULL,

    id_reserva INT NOT NULL,

    estado VARCHAR(20) DEFAULT 'aplicado', -- Permite rastrear devoluciones

    CONSTRAINT fk_uso_paquete

        FOREIGN KEY (id_usuario_paquete) REFERENCES

        usuario_paquetes(id_usuario_paquete)

        ON DELETE RESTRICT,

    CONSTRAINT fk_reserva_uso

        FOREIGN KEY (id_reserva) REFERENCES reservas(id_reserva)

        ON DELETE RESTRICT

```

);

-- =====

-- FUNCIÓN: validar cupo antes de reservar

-- =====

CREATE OR REPLACE FUNCTION validar_cupo()

RETURNS TRIGGER AS \$\$

DECLARE

 cupo_max INT;

 total_reservas INT;

BEGIN

 SELECT tc.cupo_maximo INTO cupo_max

 FROM clases c

 JOIN tipos_clase tc ON c.id_tipo = tc.id_tipo

 WHERE c.id_clase = NEW.id_clase;

 SELECT COUNT(*) INTO total_reservas

 FROM reservas

 WHERE id_clase = NEW.id_clase

 AND estado = 'activa';

 IF total_reservas >= cupo_max THEN

 RAISE EXCEPTION 'No hay cupos disponibles para esta clase';

 END IF;

```

    RETURN NEW;

END;

$$ LANGUAGE plpgsql;


CREATE TRIGGER trigger_validar_cupo
BEFORE INSERT ON reservas
FOR EACH ROW
EXECUTE FUNCTION validar_cupo();


-- =====
-- FUNCIÓN: validar uso de paquete
-- =====

CREATE OR REPLACE FUNCTION validar_paquete()
RETURNS TRIGGER AS $$
DECLARE
    clases_rest INT;
    fecha_exp DATE;
    paquete_estado VARCHAR(20);
BEGIN
    SELECT clases_restantes, fecha_expiracion, estado
    INTO clases_rest, fecha_exp, paquete_estado
    FROM usuario_paquetes
    WHERE id_usuario_paquete = NEW.id_usuario_paquete;

```



```
IF paquete_estado != 'activo' THEN
```

```
    RAISE EXCEPTION 'El paquete no se encuentra activo';
```

```
END IF;
```

```
IF clases_rest <= 0 THEN
```

```
    RAISE EXCEPTION 'No tienes clases disponibles en tu paquete';
```

```
END IF;
```

```
IF fecha_exp < CURRENT_DATE THEN
```

```
    RAISE EXCEPTION 'El paquete está vencido';
```

```
END IF;
```

```
RETURN NEW;
```

```
END;
```

```
$$ LANGUAGE plpgsql;
```

```
CREATE TRIGGER trigger_validar_paquete
```

```
BEFORE INSERT ON uso_paquete
```

```
FOR EACH ROW
```

```
EXECUTE FUNCTION validar_paquete();
```

```
-- =====
```

```
-- FUNCIÓN: descontar clase del paquete
```

```
-- =====  
  
CREATE OR REPLACE FUNCTION descontar_clase()  
RETURNS TRIGGER AS $$  
  
BEGIN  
  
    UPDATE usuario_paquetes  
  
    SET clases_restantes = clases_restantes - 1  
  
    WHERE id_usuario_paquete = NEW.id_usuario_paquete;  
  
    RETURN NEW;  
  
END;  
  
$$ LANGUAGE plpgsql;
```

```
CREATE TRIGGER trigger_descontar_clase  
AFTER INSERT ON uso_paquete  
FOR EACH ROW  
EXECUTE FUNCTION descontar_clase();
```

```
-- =====  
  
-- NUEVA FUNCIÓN: devolver clase por cancelación  
  
-- =====
```

```
CREATE OR REPLACE FUNCTION procesar_cancelacion_reserva()  
RETURNS TRIGGER AS $$  
  
DECLARE  
  
    v_id_usuario_paquete INT;
```

```

BEGIN

-- Se evalúa si el estado de la reserva fue modificado a 'cancelada'
IF NEW.estado = 'cancelada' AND OLD.estado = 'activa' THEN

    -- Se busca el paquete específico que fue utilizado para esta reserva
    SELECT id_usuario_paquete INTO v_id_usuario_paquete
    FROM uso_paquete
    WHERE id_reserva = NEW.id_reserva AND estado = 'aplicado';

    -- Si se encuentra un registro válido, se procede a la restitución
    IF FOUND THEN

        UPDATE usuario_paquetes
        SET clases_restantes = clases_restantes + 1
        WHERE id_usuario_paquete = v_id_usuario_paquete;

        UPDATE uso_paquete
        SET estado = 'revertido'
        WHERE id_reserva = NEW.id_reserva;

    END IF;

END IF;

RETURN NEW;

END;

$$ LANGUAGE plpgsql;

```

```
CREATE TRIGGER trigger_cancelacion_reserva  
AFTER UPDATE ON reservas  
FOR EACH ROW  
EXECUTE FUNCTION procesar_cancelacion_reserva();
```